

FASE 1 : COMPRENDE

1.- Visualización de Lista Enlazada:

Documento con los diagramas generados, anotados con tus propias palabras explicando:

- **1.- por qué insertar al inicio es $O(1)$:**
por que no se recorre la lista
- **2.- por qué insertar al final es $O(n)$:**
por que para insertar al final necesitas recorrer desde la cabeza hasta el ultimo nodo
- **3.- qué problema surge al eliminar el primer nodo que no ocurre con otros nodos:**
que no podemos usar la misma logica para eliminar cualquier nodo, por el hecho de que no hay un nodo anterior a la cabeza

Checkpoint Metacognitivo - Comprende

- ¿Puedo explicar en mis propias palabras por qué una lista enlazada no permite acceso $O(1)$ por índice, aunque los arreglos sí?

esto es por que las listas enlazadas están relacionadas mediante punteros

- Si me piden insertar un elemento entre el nodo 3 y el nodo 4 de una lista, ¿sé qué punteros debo modificar sin consultar código?

si, nada mas tienes que modificar el puntero el nodo anterior

- ¿Entiendo por qué necesito referencia al nodo ANTERIOR para eliminar un nodo en una lista simple?

si, debido a que si no el nodo queda "huerfano" y solo se desperdicia memoria

FASE 2: APLICA

2.- Implementación Guiada de Lista Enlazada:

Código fuente completo de tu implementación con:

- **1.- Clase Nodo:**

```
#ifndef NODO_H
#define NODO_H
class Nodo {
public:
    int dato;
    Nodo* siguiente;
    Nodo(int valor) {
        dato = valor;
        siguiente = nullptr;
    }
};

#endif
```

-
- **2.-Clase ListaEnlazada con al menos 6 métodos funcionales:**

```
#ifndef LISTAENLAZADA_H
#define LISTAENLAZADA_H

#include "Nodo.h"
#include <iostream>
using namespace std;
class ListaEnlazada {
private:
    Nodo* cabeza;    int tamaño;

    bool estaVacia() {
        return cabeza == nullptr; //1.- metodo esta vacia

    void insertarInicio(int valor) {
        Nodo* nuevoNodo = new Nodo(valor);
        nuevoNodo->siguiente = cabeza;
```

```
    cabeza = nuevoNodo;
    tamano++;
```

```
    cout << " → Insertado " << valor << " al inicio. Tamaño: " << tamano << endl;
} // 2.- metodo insertar inicio
```

```
void insertarFinal(int valor) {
    Nodo* nuevoNodo = new Nodo(valor);
```

Caso especial: Lista vacía

```
    if (estaVacia()) {
        cabeza = nuevoNodo;
        tamano++;
        cout << " → Insertado " << valor << " al final (lista vacía). Tamaño: " <<
tamano << endl;
        return;
    }
```

```
    Nodo* actual = cabeza;
    while (actual->siguiente != nullptr) {
        actual = actual->siguiente;
    }
    actual->siguiente = nuevoNodo;
    tamano++;
```

```
    cout << " → Insertado " << valor << " al final. Tamaño: " << tamano << endl;
} //3.- metodo insertar final
```

```
bool eliminarInicio() {
    if (estaVacia()) {
        cout << " Error: No se puede eliminar. La lista está vacía." << endl;
        return false;
    }
```

```
    Nodo* nodoAEliminar = cabeza;
    cabeza = cabeza->siguiente;
    int valorEliminado = nodoAEliminar->dato;
    delete nodoAEliminar;
    tamano--;
```

```
    cout << " → Eliminado " << valorEliminado << " del inicio. Tamaño: " <<
tamano << endl;
    return true;
} //4.- metodo eliminar inicio
```

```

bool buscar(int valor) {
    if (estaVacia()) {
        cout << " Búsqueda de " << valor << ": No encontrado (lista vacía)" << endl;
        return false;
    }
    Nodo* actual = cabeza;
    int posicion = 0;
    while (actual != nullptr) {
        if (actual->dato == valor) {
            cout << " Búsqueda de " << valor << ": ¡Encontrado en posición " <<
posicion << "!" << endl;
            return true;
        }
        actual = actual->siguiente;
        posicion++;
    }

    cout << " Búsqueda de " << valor << ": No encontrado" << endl;
    return false;
} // 5.- metodo buscar valor

```

```

void imprimir() {
    if (estaVacia()) {
        cout << " Lista vacía []" << endl;
        return;
    }
    cout << " Lista [tamaño=" << tamaño << "]: ";

    Nodo* actual = cabeza;
    while (actual != nullptr) {
        cout << actual->dato;
        if (actual->siguiente != nullptr) {
            cout << " -> ";
        }
        actual = actual->siguiente;
    }

    1.      cout << " -> nullptr" << endl; // 6.-metodo imprimir
}

```

- **3.- Programa de prueba que demuestre cada operación:**

```

#include <iostream>
#include "ListaEnlazada.h"
using namespace std;

void imprimirSeparador(const string& titulo) {
    cout << "\n";
    cout << "===== " << endl;
    cout << titulo << endl;
    cout << "===== " << endl;
}

void pausa() {
    cout << "\nPresiona Enter para continuar...";
    cin.ignore();
    cin.get();
}

int main() {
    cout << "\n";

    cout << "===== " << endl;
    cout << "|| DEMOSTRACIÓN: LISTA ENLAZADA SIMPLE ||" << endl;
    cout << "|| Implementación en C++ ||" << endl;
    cout << "===== " << endl;

    imprimirSeparador("SECCIÓN 1: Crear lista vacía");

    ListaEnlazada lista;

    cout << "\nVerificar si está vacía:" << endl;
    if (lista.estaVacia()) {
        cout << " ✓ La lista está vacía" << endl;
    }

    cout << "\nTamaño actual: " << lista.obtenerTamanio() << endl;

    cout << "\nIntentando imprimir lista vacía:" << endl;
    lista.imprimir();

    imprimirSeparador("SECCIÓN 2: Insertar al inicio");

    cout << "\nInsertando elementos: 10, 20, 30, 40" << endl;
    lista.insertarInicio(10);

```

```
lista.insertarInicio(20);  
lista.insertarInicio(30);  
lista.insertarInicio(40);
```

```
cout << "\nEstado de la lista:" << endl;  
lista.imprimir();  
cout << "Tamaño: " << lista.obtenerTamano() << endl;
```

```
cout << "\n Nota: Los elementos se insertan en orden INVERSO" << endl;  
cout << " porque cada nuevo elemento se coloca al INICIO." << endl;
```

```
imprimirSeparador("SECCIÓN 3: Insertar al final");
```

```
cout << "\nInsertando elementos: 5, 15, 25" << endl;  
lista.insertarFinal(5);  
lista.insertarFinal(15);  
lista.insertarFinal(25);
```

```
cout << "\nEstado de la lista:" << endl;  
lista.imprimir();  
cout << "Tamaño: " << lista.obtenerTamano() << endl;
```

```
imprimirSeparador("SECCIÓN 4: Buscar elementos");
```

```
cout << "\nBuscando elementos existentes:" << endl;  
lista.buscar(40);  
lista.buscar(20);  
lista.buscar(25);
```

```
cout << "\nBuscando elementos que NO existen:" << endl;  
lista.buscar(100);  
lista.buscar(-5);
```

```
imprimirSeparador("SECCIÓN 5: Eliminar del inicio");
```

```
cout << "\nLista antes de eliminar:" << endl;  
lista.imprimir();
```

```
cout << "\nEliminando 3 elementos del inicio:" << endl;  
lista.eliminarInicio();  
lista.eliminarInicio();  
lista.eliminarInicio();
```

```

    cout << "\nLista después de eliminar:" << endl;
    lista.imprimir();
    cout << "Tamaño: " << lista.obtenerTamanio() << endl;
    imprimirSeparador("SECCIÓN 6: Operaciones combinadas");

    cout << "\nInsertar al inicio (100):" << endl;
    lista.insertarInicio(100);
    lista.imprimir();

    cout << "\nInsertar al final (200):" << endl;
    lista.insertarFinal(200);
    lista.imprimir();

    cout << "\nEliminar del inicio:" << endl;
    lista.eliminarInicio();
    lista.imprimir();

    cout << "\nBuscar elemento 200:" << endl;
    lista.buscar(200);
    imprimirSeparador("SECCIÓN 7: Vaciar la lista");

    cout << "\nEliminando todos los elementos:" << endl;
    cout << "Tamaño inicial: " << lista.obtenerTamanio() << endl;

    while (!lista.estaVacia()) {
        lista.eliminarInicio();
    }

    cout << "\nEstado final:" << endl;
    lista.imprimir();
    cout << "Tamaño: " << lista.obtenerTamanio() << endl;

    cout << "\nIntentando eliminar en lista vacía:" << endl;
    lista.eliminarInicio();

    imprimirSeparador("SECCIÓN 8: Nueva lista de prueba");

    cout << "\nCreando lista: [1, 2, 3, 4, 5]" << endl;
    lista.insertarFinal(1);
    lista.insertarFinal(2);
    lista.insertarFinal(3);
    lista.insertarFinal(4);
    lista.insertarFinal(5);
    lista.imprimir();

```

```
imprimirSeparador("SECCIÓN 9: Casos especiales");
```

```
cout << "\nCASO 1: Buscar en lista con elementos:" << endl;
lista.buscar(3);
lista.buscar(99);
```

```
cout << "\nCASO 2: Eliminar hasta tener un solo elemento:" << endl;
while (lista.obtenerTamanio() > 1) {
    lista.eliminarInicio();
}
lista.imprimir();
```

```
cout << "\nCASO 3: Operaciones con un solo elemento:" << endl;
lista.buscar(5);
lista.insertarInicio(10);
lista.imprimir();
```

```
cout << "\nCASO 4: Insertar valores duplicados:" << endl;
lista.insertarFinal(10);
lista.insertarFinal(10);
lista.imprimir();
cout << "Buscar valor duplicado (encuentra el primero):" << endl;
lista.buscar(10);
```

```
imprimirSeparador("SECCIÓN 10: Resumen");
```

```
cout << "\n ESTADÍSTICAS FINALES:" << endl;
cout << " • Tamaño de la lista: " << lista.obtenerTamanio() << endl;
cout << " • ¿Está vacía?: " << (lista.estaVacia() ? "Sí" : "No") << endl;
cout << "\n Estado actual:" << endl;
lista.imprimir();
```

```
cout << "\n COMPLEJIDADES DE LAS OPERACIONES:" << endl;
cout << " • estaVacia()      : O(1)" << endl;
cout << " • insertarInicio()   : O(1) " << endl;
cout << " • insertarFinal()    : O(n)" << endl;
cout << " • eliminarInicio()   : O(1) " << endl;
cout << " • buscar()           : O(n)" << endl;
cout << " • imprimir()         : O(n)" << endl;
cout << " • obtenerTamanio()   : O(1)" << endl;
```

```
cout << "\n CARACTERÍSTICAS DE LISTAS ENLAZADAS:" << endl;
cout << " Inserción/eliminación al inicio muy eficiente" << endl;
```



```
cout << " Tamaño dinámico (crece según necesidad)" << endl;
cout << " No requiere memoria contigua" << endl;
cout << " Acceso secuencial (no hay índices)" << endl;
cout << " Mayor uso de memoria (por los punteros)" << endl;
```

```
imprimirSeparador("FIN DE LA DEMOSTRACIÓN");
```

```
cout << "\n ¡Todas las operaciones ejecutadas exitosamente!" << endl;
cout << "\nEl destructor se ejecutará automáticamente al salir..." << endl;
cout << "(Observa el mensaje de liberación de memoria)\n" << endl;
```

```
return 0;
```

- **4.- Comentarios explicando la lógica de cada método.:**

Método 1 listaenlazada():

La función de este método es preparar el objeto para su uso, estableciendo la cabeza como nullptr y el tamaño en cero. Esto nos garantiza que la lista comience en un estado válido y vacío.

Método 2 estavacia():

El propósito de este método es verificar si la lista contiene elementos o no. Internamente compara si cabeza apunta a nullptr. No modifica nada en la lista, solo observa su estado actual.

Método 3 insertarInicio(valor):

Su función es crear un nuevo nodo en memoria dinámica y colocarlo al frente de la lista, actualizando la cabeza para que apunte a este nuevo nodo, lo que hace que el nodo anterior se convierta en el segundo elemento.

Método 4 insertarFinal(valor):

Lo que hace primero es validar que la lista no esté vacía para evitar errores, luego guarda una referencia al nodo a eliminar, después mueve la cabeza al segundo nodo, y luego libera la memoria del primer nodo usando delete.

Método 5 buscar(valor):

Lo que hace este método es recorrer la lista secuencialmente desde la cabeza, comparando cada nodo con el valor buscado. Si lo encuentra, termina de inmediato y retorna true, sin revisar el resto de los nodos. Si llega al final sin encontrarlo, retorna false. Es una búsqueda lineal que puede ser rápida si el elemento está al inicio o lenta si está al final.

Metodo 6 imprimir():

Este método utiliza un puntero temporal para recorrer todos los nodos desde la cabeza hasta nullptr, imprimiendo el valor de cada uno seguido de una flecha. No modifica la estructura ni los datos, solo los lee y muestra.

3.- Resolver Problemas Clásicos con Depuración Visual:

Implementación de al menos 2 problemas clásicos (invertir lista + encontrar medio) con:

- **1.- Código funcional probado:**

```
#include <cassert>
#include <iostream>
#include <vector>

// =====
// Estructura de Nodo
// =====
struct Node {
    int val;
    Node* next;
    explicit Node(int v) : val(v), next(nullptr) {}
};

// =====
// Utilidades de lista
// =====
Node* buildList(const std::vector<int>& a) {
    Node* head = nullptr;
    Node* tail = nullptr;
    for (int x : a) {
        Node* n = new Node(x);
        if (!head) head = tail = n;
        else { tail->next = n; tail = n; }
    }
    return head;
}

std::vector<int> toVector(Node* head) {
    std::vector<int> out;
    for (Node* cur = head; cur != nullptr; cur = cur->next)
        out.push_back(cur->val);
}
```

```

    return out;
}

void freeList(Node* head) {
    while (head) {
        Node* nxt = head->next;
        delete head;
        head = nxt;
    }
}

// =====
// Problema 1: Invertir lista (iterativo, 3 punteros)
// =====
Node* reverseList(Node* head) {
    Node* anterior = nullptr;
    Node* actual = head;

    while (actual != nullptr) {
        Node* siguiente = actual->next; // 1) guardar el resto
        actual->next = anterior;        // 2) invertir el enlace
        anterior = actual;              // 3) avanzar anterior
        actual = siguiente;             // 4) avanzar actual
    }
    return anterior; // nuevo head
}

// =====
// Problema 2: Encontrar el nodo medio (slow/fast pointers)
// - Si longitud es impar: retorna el medio exacto
// - Si longitud es par: retorna el "segundo medio" (por convención común)
// Ej: 1->2->3->4 retorna 3
// =====
Node* middleNode(Node* head) {
    if (!head) return nullptr;

    Node* slow = head;
    Node* fast = head;

    // fast avanza 2, slow avanza 1.
    // Cuando fast llega al final, slow queda en el medio.
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }
}

```

```

    }
    return slow;
}

// =====
// Tests básicos (assert)
// =====
void testReverse() {
    {
        Node* head = buildList({}); // lista vacía
        Node* r = reverseList(head);
        assert(r == nullptr);
    }
    {
        Node* head = buildList({1});
        Node* r = reverseList(head);
        assert(toVector(r) == std::vector<int>({1}));
        freeList(r);
    }
    {
        Node* head = buildList({1,2,3,4});
        Node* r = reverseList(head);
        assert(toVector(r) == std::vector<int>({4,3,2,1}));
        freeList(r);
    }
}

void testMiddle() {
    {
        Node* head = buildList({});
        assert(middleNode(head) == nullptr);
    }
    {
        Node* head = buildList({10});
        assert(middleNode(head)->val == 10);
        freeList(head);
    }
    {
        Node* head = buildList({1,2,3,4,5});
        assert(middleNode(head)->val == 3);
        freeList(head);
    }
    {
        Node* head = buildList({1,2,3,4});

```

```

        // segundo medio por convención -> 3
        assert(middleNode(head)->val == 3);
        freeList(head);
    }
}

int main() {
    testReverse();
    testMiddle();
    std::cout << "OK: todos los tests pasaron.\n";
    return 0;
}

```

- **2.- Documento explicando el algoritmo paso a paso en tus propias palabras:**

-

1.- Invertir una lista enlazada:

El objetivo es hacer que cada nodo apunte “hacia atrás” en vez de hacia adelante.

El reto es no perder el acceso al resto de la lista mientras vamos cambiando punteros.

Para eso tendremos que usar 3 punteros:

1 anterior: representa el inicio de la parte que ya quedó invertida.

2.actual: el nodo que estamos procesando en este momento.

3. siguiente: guarda temporalmente el resto de la lista que falta (para no perderlo).

En cada vuelta del ciclo (while) hacemos 4 acciones:

1. Guardar el resto

siguiente = actual->next

esto guarda “a dónde ibamos” antes de romper el enlace.

2. Invertir el enlace del nodo actual

actual->next = anterior

Ahora `actual` apunta hacia la parte invertida.

3. Mover anterior hacia adelante

anterior = actual

lo invertido crece: el nuevo inicio es actual.

4. Avanzar actual a lo pendiente
actual = siguiente
pasamos al próximo nodo original.

Cuando termina el ciclo, `actual` es `null` y `anterior` quedó apuntando al nuevo inicio de la lista ya invertida.

2. Encontrar el nodo medio

Tenemos que usar dos punteros:

slow avanza de 1 en 1

fast avanza de 2 en 2

Mientras fast pueda seguir avanzando 2 pasos, seguimos iterando:

slow = slow->next

fast = fast->next->next

El código funciona porque fast recorre la lista al doble de velocidad y para cuando fast llega al final, slow habrá recorrido la mitad.

Pero ahora digamos que en una lista con longitud par hay “dos medios” (ej: 1-2-3-4). Si es así entonces con el bucle while (fast && fast->next), slow termina en el segundo medio (3 en el ejemplo). Esto es una convención frecuente (por ejemplo, la de muchos ejercicios tipo LeetCode).

● **3.- Análisis de complejidad temporal y espacial:**

1. Invertir lista

Este problema tiene la complejidad $O(n)$ ya que recorremos cada nodo exactamente una vez y hacemos operaciones constantes por nodo y la complejidad espacial es $O(1)$ debido a que solo usamos unos pocos punteros (anterior, actual, siguiente).

2.- Encontrar el medio

su complejidad es $O(n)$ ya que aunque fast salta de 2 en 2, el número de iteraciones es proporcional a $n/2$, que sigue siendo $O(n)$ y su complejidad espacial es $O(1)$ ya que solo usamos slow y fast.

Checkpoint Metacognitivo - Aplica

- ¿Puedo implementar insertarInicio() sin mirar el código? ¿Qué línea escribiría primero y por qué en ese orden?
sí, nada más sería crear un nodo nuevo, apuntarlo a lo que era la cabeza antiguamente y actualizarlo

- Si mi método eliminar() no funciona para el primer elemento, ¿puedo identificar qué caso especial olvidé?
sí, y eso pasa cuando el nodo eliminar es la cabeza
- ¿Cuánto del código escribí yo versus cuánto copié directamente de la IA sin modificar?
como un 15%, estoy tratando de primero comprender lo que hace la ia para luego tratar de hacerlo por mi cuenta

FASE 3: REFLEXIONA

4.- Análisis de Casos de Uso Reales

Documento comparativo que incluya:

1. Tu análisis inicial de cada escenario ANTES de consultar la IA:

A) Sistema de playlist de música:

escenario 1: insertar al final

escenario 2: insertarEn(pos, valor)

escenario 3: obtener(posicion)

escenario 4: eliminarEn(pos)

escenario 5: tamaño()

B) Buffer de teclado:

escenario 1: insertarInicio(valor)

escenario 2: imprimir()

escenario 3: tamaño()

escenario 4: nueva()

C) Sistema de versiones de documento (como Git):

escenario 1: Arreglo()

escenario 2: buscar(valor)

escenario 3: buscar(valor)

escenario 4: obtener(posicion)

2. El análisis de la IA:

A) Sistema de playlist de música:

escenario 1: Agregar al final

escenario 2: Insertar en posición

escenario 3: Acceso aleatorio

escenario 4: Eliminación en posición

escenario 5: Navegación secuencial

B) Buffer de teclado:

escenario 1: Enqueue

escenario 2: Dequeue

escenario 3: Búsqueda

escenario 4: Tamaño fijo

C) Sistema de versiones de documento (como Git):

escenario 1: Append

escenario 2: Acceso al HEAD

escenario 3: Navegación hacia atrás

escenario 4: Sin eliminaciones

3. Tu reflexión sobre las diferencias entre ambos análisis:

la reflexión que puedo generar es que la IA tuvo un enfoque distinto al mío, yo lo que hice fue basarme en las operaciones que venían en el archivo de la semana tres y sin embargo la IA respondió con otro tipo de operaciones, aunque algunas sí son similares a las vistas en el archivo de la semana 3

4. Conclusiones sobre cuándo usar cada estructura:

evidentemente, cada estructura se utiliza para diferentes situaciones, por lo que puedo concluir que no hay ninguna mejor que la otra, sino que depende de la situación en la que estes, las listas se recomiendan usar cuando el tamaño es impredecible y esta constantemente creciendo y los arreglos son más para cuando conoces los límites de tus datos

Checkpoint Metacognitivo - Reflexiona

- Si alguien me pregunta "¿es mejor usar arreglo o lista enlazada?", ¿puedo responder "depende" y explicar de qué depende?
sí, ya que ninguna es mejor que otra si no que los arreglos sirven más para datos pequeños o que los conoces bien por así decir y las listas sirven más para cuando no sabes que tanto puede crecer los datos

- ¿Puedo identificar al menos 2 escenarios donde elegir la estructura incorrecta causaría problemas graves de rendimiento?
si, por ejemplo causaria problemas usar una lista enlazada con datos pequeños como unos 500 asi como estaria mal usar un arreglo con mas de 1000000 datos
- Cuando la IA me dio una recomendación, ¿la acepté automáticamente o cuestioné su razonamiento?
no me parecio nada que pudiera cuestionar

FASE 4: VALIDA

5.- Validación de Implementación con Técnica de "Rubber Duck"

Transcripción de la sesión de "rubber duck debugging" con al menos 3 métodos diferentes. Para cada método, incluye:

1. El código original:

1.- insertarinicio:

```
void insertarInicio(int valor) {
    Nodo* nuevoNodo = new Nodo(valor);

    nuevoNodo->siguiente = cabeza;

    cabeza = nuevoNodo;

    tamano++;

    cout << "Insertado " << valor << " al inicio. Tamaño: " << tamano << endl;
}
```

2.- eliminarinicio:

```
bool eliminarInicio() {
    if (estaVacia()) {
```

```

        cout << "Error: No se puede eliminar. La lista está vacía." << endl;
        return false;
    }

    Nodo* nodoAEliminar = cabeza;

    cabeza = cabeza->siguiente;

    int valorEliminado = nodoAEliminar->dato;

    delete nodoAEliminar;

    tamano--;

    cout << "Eliminado " << valorEliminado << " del inicio. Tamaño: " << tamano <<
endl;
    return true;    }

```

3.-buscar:

```

Nodo* actual = cabeza;
int posicion = 0;

while (actual != nullptr) {

    if (actual->dato == valor) {
        cout << "Búsqueda: " << valor << " - ¡Encontrado en posición " << posicion
<< "!" << endl;
        return true;
    }

    actual = actual->siguiente;
    posicion++;
}

cout << "Búsqueda: " << valor << " - No encontrado" << endl;
return false;
}

```

2. Las preguntas de la IA y tus respuestas:

1.- insertar inicio:

Pregunta 1: La palabra clave new

Nodo* nuevoNodo = new Nodo(valor);

Pregunta: ¿Qué hace exactamente la palabra clave new?

respuesta: new reserva memoria en el heap

Pregunta 2: Dónde apunta siguiente

nuevoNodo->siguiente = cabeza;

Pregunta: Después de esta línea, ¿a dónde está apuntando nuevoNodo->siguiente?

respuesta: Está apuntando a donde apunta cabeza

Pregunta 3: Por qué este orden

Pregunta: ¿Qué pasaría si cambiaras el orden y primero hicieras cabeza = nuevoNodo y DESPUÉS nuevoNodo->siguiente = cabeza?

respuesta: Se perdería la referencia al primer nodo

Pregunta 4: Actualizar cabeza

cabeza = nuevoNodo;

Pregunta: ¿Qué está pasando aquí? ¿Por qué necesitas actualizar cabeza?

respuesta: Para que apunte al nuevo primer nodo

Pregunta 5: Incrementar tamaño

tamano++;

Pregunta: ¿Qué pasaría si te olvidas de esta línea? El nodo físicamente Sí se inserta, pero ¿qué quedaría mal?

respuesta:

No se incrementa

Pregunta 6: Complejidad temporal

Pregunta: ¿Cuál es la complejidad de insertarInicio y insertarFinal?

respuesta:

Insertar al inicio es $O(1)$ y Insertar al final es $O(n)$

2.- eliminarInicio:

Pregunta 1: Verificación inicial

```
if (estaVacia()) {  
    cout << "Error: No se puede eliminar. La lista está vacía." << endl;  
    return false;  
}
```

Pregunta: ¿Por qué es necesario verificar si la lista está vacía?

respuesta: Eso es para evitar crash

Pregunta 2: Guardar referencia

```
Nodo* nodoAEliminar = cabeza;
```

Pregunta: ¿Por qué guardas cabeza en otra variable?

respuesta: Porque vamos a cambiar la cabeza en el siguiente paso

Pregunta 3: Actualizar cabeza

```
cabeza = cabeza->siguiente;
```

Pregunta: ¿A qué está apuntando cabeza después de esta línea? ¿Y si solo había un nodo?

respuesta: Está apuntando al segundo nodo o a nullptr si solo hay un nodo

Pregunta 4: Guardar el valor

```
int valorEliminado = nodoAEliminar->dato;
```

Pregunta: ¿Por qué guardas el valor ANTES de hacer delete?

respuesta: Eso es para obtener el valor antes de eliminarlo

Pregunta 5: Delete y punteros

```
delete nodoAEliminar;
```

Pregunta: ¿Qué le pasa al puntero nodoAEliminar después de delete?

respuesta: Sigue teniendo la dirección pero apunta a memoria liberada (dangling pointer)

Pregunta 6: Decrementar tamaño

```
tamanio--;
```

Pregunta: ¿Qué pasaría si olvidas esta línea?

respuesta: No se decrementa

Pregunta 7: Orden crítico

Pregunta: ¿Qué pasaría si hicieras delete nodoAEliminar PRIMERO y luego cabeza = cabeza->siguiente?

respuesta: Si haces delete primero liberas la memoria

3.- buscar(valor):

Pregunta 1: Variables iniciales

Nodo* actual = cabeza;

int posicion = 0;

Pregunta: ¿Por qué usas un puntero actual en lugar de trabajar directamente con cabeza? ¿Y para qué necesitas posicion?

respuesta: El puntero actual es para recorrer la lista. Sobre la posición es el contador para reportar dónde se encontró el elemento

Pregunta 2: Condición del while

while (actual != nullptr) {

Pregunta: ¿Por qué la condición es actual != nullptr? ¿Qué significa cuando actual llega a ser nullptr?

respuesta: La condición se refiere a que mientras haya nodos por revisar se debe continuar

Pregunta 3: Return inmediato

if (actual->dato == valor) {

cout << "Búsqueda: " << valor << " - ¡Encontrado en posición " << posicion << "!" << endl;

return true;

}

Pregunta: ¿Qué pasa con el resto del while cuando encuentras el valor? ¿Y si el valor aparece múltiples veces?

respuesta: El while hace una salida inmediata al encontrar el valor, lo que significa que este se detiene. Si aparece el valor múltiples veces solo se detiene en el primer elemento

Pregunta 4: Avanzar en la lista

actual = actual->siguiente;

posicion++;

Pregunta: ¿Qué hace cada línea? ¿Por qué se ejecutan?

respuesta: La primera hace que si no encontramos el valor que buscamos, sigamos buscando valores. La línea de posición lo que hace es contar las posiciones que avanzamos

Pregunta 5: Después del while

**out << "Búsqueda: " << valor << " - No encontrado" << endl;
return false;**

Pregunta: ¿En qué situación se ejecutan estas líneas?

respuesta: Se ejecutan cuando el valor que encontramos no está en la lista